

DATA ENGINEERING ADVANCED

Exercice pratique

Construire un Data Lake E-commerce sur AWS

Durée cible	Environ 5 jours
Niveau	Accessible en autonomie — synthèse des modules 0 à 5
Prérequis	Modules 1 à 5 : environnement Cloud, IAM, Budget, Terraform / CloudFormation / CloudWatch, S3 + Athena
Livrable final	Infrastructure Terraform + Data Lake médaillon + modèle dimensionnel exploitable en BI

Ce document est l'énoncé remis aux participants. Les sources de données (orders.csv, products.jsonl, users.jsonl) sont fournies séparément dans le dossier sources/.

1. Cas d'usage

Vous rejoignez l'équipe Data d'une entreprise e-commerce de taille moyenne. La DSI impose que toute infrastructure soit provisionnée en Terraform — pas d'action manuelle dans la console — avec une gouvernance IAM stricte, un budget maîtrisé et un monitoring en place. C'est exactement ce que vous avez mis en place dans les modules précédents.

Votre mission : construire le Data Lake de l'entreprise à partir de deux sources réelles de nature différente — un export de commandes issu du système de vente (CSV) et le catalogue produits/clients de l'application (API JSON) — puis livrer un résultat exploitable par l'équipe Marketing/BI.

L'équipe BI ne veut pas d'une grosse table plate : elle veut un modèle en étoile simple (faits + dimensions), branchable directement sur un outil de reporting, capable de répondre par exemple à :

« Quel est le chiffre d'affaires par pays et par catégorie de produit sur les 3 derniers mois, et quelles commandes référencent des clients ou produits qui n'existent plus dans le catalogue applicatif ? »

2. Questions analytiques à répondre

Votre zone **gold** (schéma en étoile) doit permettre de répondre, par simple requête Athena (jointures fact/dimensions), aux questions suivantes posées par le Marketing/BI. Elles ne sont pas toutes du même niveau de difficulté — certaines nécessitent d'avoir correctement traité les anomalies de la zone silver pour donner un résultat juste.

#	Question métier
1	Quel est le chiffre d'affaires total, et par pays, sur les 3 derniers mois ?
2	Quels sont les 10 produits qui génèrent le plus de chiffre d'affaires, et les 10 produits les plus vendus en quantité ? Est-ce les mêmes ?
3	Comment le chiffre d'affaires et le nombre de commandes évoluent-ils mois par mois ?
4	Quel est le panier moyen (montant moyen par commande) par pays ?
5	Quels sont les 5 clients qui ont généré le plus de chiffre d'affaires cumulé ?
6	Combien de lignes de commande référencent un produit ou un client absent du catalogue applicatif (clé orpheline), et quel volume/CA cela représente-t-il ? Comment ces lignes sont-elles traitées dans votre modèle (exclues, isolées, rattachées à une clé « inconnu ») ?

Clé orpheline : une valeur de ProductID/CustomerID présente dans orders.csv mais absente du catalogue applicatif (products.json/users.json) — par exemple un produit retiré du catalogue ou un compte client supprimé après la commande. Une jointure classique (INNER JOIN) fait disparaître ces lignes en silence ; l'enjeu est de les détecter et de décider d'un traitement explicite plutôt que de perdre du chiffre d'affaires sans s'en apercevoir.

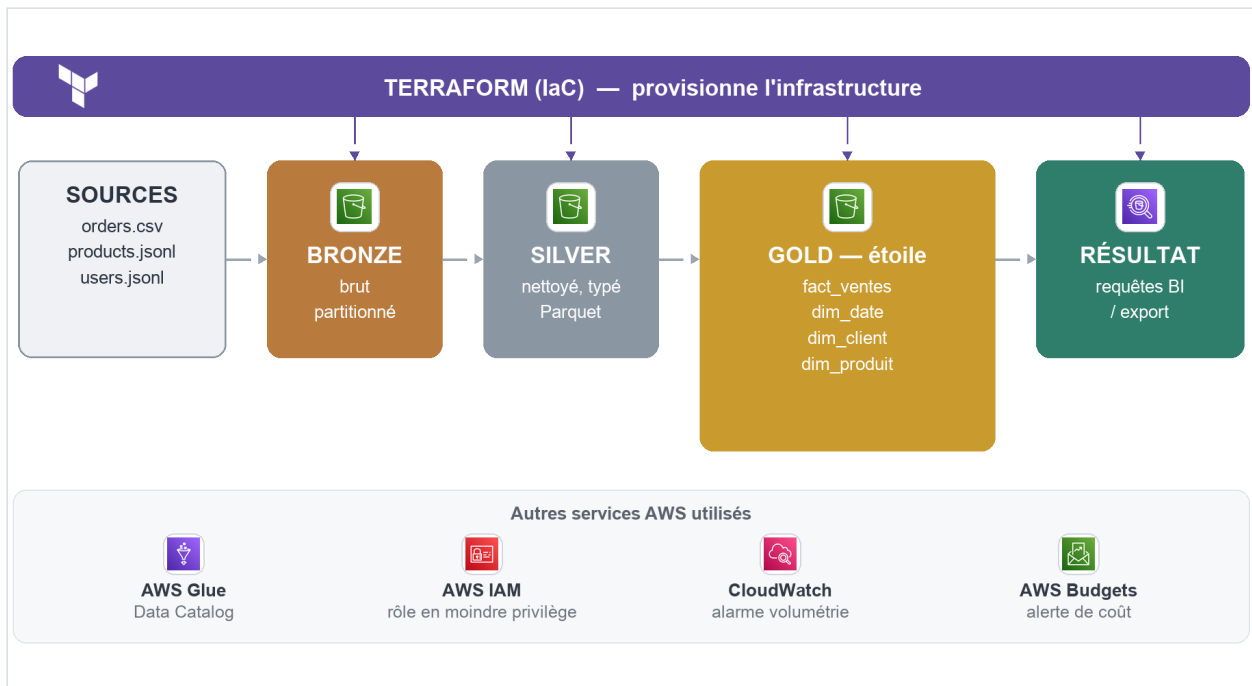
Préparez vos requêtes et vos résultats pour votre session live avec le coach — l'idée est de les présenter et d'en discuter à l'oral, pas de les envoyer par écrit pour relecture.

ALLER PLUS LOIN (optionnel)

Si vous avez fini ces 6 questions avant le reste du groupe, deux extensions possibles (à discuter avec votre coach) : la répartition du chiffre d'affaires par catégorie de produit et son évolution mois par mois, et le taux de retour (commandes annulées) par catégorie.

3. Architecture AWS proposée

L'architecture suit une logique de **zones de maturité (médaillon)** : **bronze** (données brutes, fidèles à la source), **silver** (nettoyées, typées, déduplicuées, au format Parquet partitionné) et **gold** (modèle dimensionnel prêt pour la BI). L'ensemble de l'infrastructure est provisionné par Terraform.



- **Bronze** : fidèle à la source, aucune transformation — traçabilité et possibilité de rejouer le pipeline.
- **Silver** : nettoyée, typée, déduplicuée, Parquet partitionné.
- **Gold** : modèle dimensionnel (une table de faits + ses dimensions), consommable directement par la BI.
- La cataloguage se fait via des instructions SQL Athena (`CREATE EXTERNAL TABLE`, `CTAS`), cohérent avec le module 5 — sans introduire de nouveau service de cataloguage automatisé.

4. Structure des tables attendue

Cette section décrit la structure cible de chaque zone : elle vous guide sur le *quoi* (colonnes, clés), pas sur le *comment* (DDL, règles de nettoyage exactes, ordre des opérations) — cela reste à concevoir. Les noms de colonnes sont indicatifs ; gardez simplement des noms clairs et cohérents entre les zones.

Bronze — telle quelle, aucun typage strict

orders_raw (CSV, une ligne = une ligne de commande) :

Colonne	Type
InvoiceNo	string
ProductID	string
Quantity	string
InvoiceDate	string
UnitPrice	string
CustomerID	string
Country	string

products_raw et users_raw (JSON, un objet par ligne) : structure imbriquée conservée telle quelle (dimensions, tags côté produit ; address, company côté client). Tout typer en STRING en bronze n'est pas obligatoire pour le JSON imbriqué (structs/arrays typés dès la lecture), mais évitez de forcer des types numériques stricts sur le CSV brut — une valeur invalide ferait échouer la lecture de toute la table.

Silver — nettoyée, typée, un nom de table par source

Table	Colonnes clés à produire
orders_clean	invoiceno, product_id (num.), quantity (num.), unit_price (num.), line_amount, customer_id (num., nullable), country (standardisé), is_return (bool), invoice_date (date)
products_clean	product_id, title, category, brand, catalog_price, discounted_price, stock, largeur/hauteur/profondeur aplaties depuis dimensions
users_clean	customer_id, firstname, lastname, email, city, country, company_name (aplatis depuis company)

À vous de définir : le format Parquet, le partitionnement, les règles d'exclusion (date illisible, prix nul...), et la standardisation des pays (plusieurs graphies coexistent dans orders.csv).

Gold — schéma en étoile

Grain de fact_ventes : une ligne = un produit vendu dans une commande.

Table	Colonnes	Rôle
dim_date	date_id (PK), full_date, year, month, month_name, day, quarter	Dimension temps
dim_produit	product_id (PK), title, category, brand, catalog_price, discounted_price	Dimension produit (+ 1 ligne conventionnelle pour les clés orphelines, cf. Q6)

Table	Colonnes	Rôle
dim_client	customer_id (PK), firstname, lastname, email, city, country, company_name	Dimension client (+ 1 ligne conventionnelle pour les clés orphelines/manquantes)
fact_ventes	invoiceno, date_id (FK), product_id (FK), customer_id (FK), country, quantity, unit_price, line_amount, is_return	Table de faits — mesures : quantity, unit_price, line_amount

PK = clé primaire de la dimension, FK = clé étrangère dans la table de faits. Le pays est porté directement par fact_ventes (c'est le pays de livraison déclaré sur la commande, pas un attribut du client) plutôt que d'en faire une dimension séparée — un choix de simplification assumé, pas une obligation. Le mécanisme de rattachement des clés orphelines/manquantes (quel identifiant utiliser, comment le documenter) est à votre initiative — c'est un des points évalués (question 6, critère « modèle dimensionnel »).

ALLER PLUS LOIN (optionnel)

Si le modèle à 3 dimensions vous semble trop simple, vous pouvez extraire le pays dans une vraie dimension dim_pays séparée (jointure depuis fact_ventes plutôt qu'attribut direct) — c'est le choix fait dans la plupart des schémas en étoile « classiques ». À discuter avec votre coach avant de vous lancer.

Indices pour démarrer

INDICE — Infrastructure (Terraform)	
Ressources clés	<code>aws_s3_bucket</code> , <code>aws_iam_role</code> + <code>aws_iam_role_policy</code> , <code>aws_glue_catalog_database</code> , <code>aws_budgets_budget</code> , <code>aws_cloudwatch_metric_alarm</code> , <code>aws_sns_topic</code> + <code>aws_sns_topic_subscription</code> .
Nom du bucket	S3 = espace de noms global à l'échelle de tout AWS (tous comptes, toutes régions confondus) : le nom doit être évidemment unique , pas juste unique dans votre compte. Combinez un préfixe de projet avec un data <code>aws_caller_identity</code> ou une ressource <code>random_id</code> pour éviter toute collision.
Région AWS	Choisissez la région (<code>aws_region</code>) géographiquement la plus proche de vous — moins de latence sur vos requêtes Athena et vos uploads S3, et c'est aussi une bonne pratique à justifier dans votre documentation.
Chiffrement & accès S3	Pas besoin de déclarer explicitement le chiffrement ni le blocage d'accès public : depuis 2023, un bucket S3 nouvellement créé a le chiffrement SSE-S3 et le Block Public Access activés par défaut sur un compte AWS standard.
Pas de workgroup dédié	Utilisez le workgroup <code>primary</code> (toujours présent par défaut) plutôt que d'en créer un : ça évite la mécanique <code>enforce_workgroup_configuration/external_location</code> — un vrai piège si vous créez un workgroup dédié avec une configuration de sortie fixe, incompatible avec l' <code>external_location</code> de vos CTAS silver/gold. Chaque requête précisera plutôt son propre emplacement de résultat.
Policy IAM	Écrivez-la avec un data <code>aws_iam_policy_document</code> plutôt qu'un bloc JSON en dur dans une string : plus lisible, moins d'erreurs de syntaxe.
AWS Budgets	Peut contenir plusieurs blocs notification (un par seuil d'alerte).
Alarme CloudWatch	Référence un <code>aws_sns_topic</code> via <code>alarm_actions</code> .
Confirmation SNS	Une souscription par email reste PENDING tant que le destinataire n'a pas cliqué le lien reçu — pensez-y avant de considérer l'alarme « fonctionnelle ».
Pas de valeur en dur	Passez par <code>variables.tf</code> / <code>outputs.tf</code> — c'est un des critères d'évaluation.
<code>terraform destroy</code> propre	Un bucket contient toujours des objets en fin d'exercice : sans <code>force_destroy = true</code> sur <code>aws_s3_bucket</code> , <code>terraform destroy</code> échoue avec « BucketNotEmpty ». Il faut un <code>terraform apply</code> pour écrire ce paramètre dans le state <i>avant</i> qu'un <code>destroy</code> en tienne compte.

INDICE — Athena / SQL

CTAS	<code>CREATE TABLE ... WITH (format='PARQUET', external_location='s3://...', partitioned_by=ARRAY[...]) AS SELECT ...</code> — les colonnes de partition doivent être les dernières de la liste SELECT.
JSON imbriqué	La SerDe <code>org.openx.data.jsonserde.JsonSerDe</code> type directement les champs imbriqués en <code>struct<...>/array<...></code> , sans les aplatir dès le bronze.
Nouvelles partitions	Invisibles tant qu'on n'a pas exécuté <code>MSCK REPAIR TABLE</code> après un dépôt S3.
Typage tolérant	<code>TRY_CAST(col AS type)</code> et <code>TRY(date_parse(...))</code> renvoient NULL plutôt que de faire échouer toute la requête sur une seule valeur invalide.
Détecter les doublons	<code>GROUP BY</code> sur l'ensemble des colonnes + <code>HAVING COUNT(*) > 1</code> pour compter combien de lignes sont des doublons exacts avant de les éliminer — ne devinez pas, mesurez.
Éliminer les doublons	<code>SELECT DISTINCT</code> suffit pour des doublons exacts (toutes les colonnes identiques). Pour ne garder qu'une ligne parmi des quasi-doublons (mêmes clés métier, une valeur différente), utilisez <code>ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)</code> dans un CTE puis filtrez <code>WHERE rn = 1</code> .
Clés orphelines	<code>CASE WHEN ... NOT IN (SELECT ... FROM dim_...)</code> pour les rattacher à une clé « inconnu ».

5. Services AWS utilisés et rôle de chacun

Service	Rôle dans le projet
Terraform + Amazon Athena	Terraform provisionne l'infrastructure (bucket S3 bronze/silver/gold, rôle et politiques IAM, AWS Budget, alarme CloudWatch) — rien à provisionner pour Athena, qui tourne sur le workgroup « primary » par défaut. Athena sert ensuite à cataloguer et transformer les données en SQL (DDL, CTAS bronze → silver → gold, requêtes BI finales).
Amazon S3	Stockage du Data Lake, organisé en 3 zones avec une convention de nommage/partitionnement.
AWS IAM	Rôle dédié au projet en moindre privilège — accès scopé au bucket et aux préfixes nécessaires, pas de credentials personnels dans les scripts.
AWS Budgets	Alerte de coût sur le projet (seuil défini), pour éviter un dépassement pendant la semaine d'exercice.
Amazon CloudWatch	Alarme sur la taille du bucket, pour surveiller que la volumétrie reste dans l'attendu.

Sources de données fournies

Fichier	Description
sources/bronze_csv/orders.csv	Export « ERP » des commandes (~8 000 lignes). Contient des anomalies réelles à détecter : doublons, valeurs manquantes, identifiants orphelins, formats de pays incohérents, dates invalides, prix aberrants. Aucune liste d'anomalies n'est fournie — la détection fait partie de l'exercice.
sources/bronze_json/products.jsonl	Catalogue produit (source applicative, 130 produits), un objet JSON par ligne, structure imbriquée (dimensions, tags).
sources/bronze_json/users.jsonl	Clients (source applicative, 130 clients), un objet JSON par ligne, structure imbriquée (adresse, entreprise).

Un squelette Terraform de démarrage est fourni dans `demarrage_terraform/` : de quoi démarrer le Jour 1 sans perdre de temps sur la mise en page du projet plutôt que sur les choix d'architecture.

Fichiers Terraform à réaliser

Fichier	Description et but
main.tf	Le cœur du provisioning : bucket S3 (le Data Lake), base Glue Data Catalog, AWS Budget, alarme CloudWatch + topic SNS. Tout ce qui n'est pas de la gouvernance/sécurité. Bloc <code>terraform{} / provider</code> et plomberie déjà fournis dans le squelette — ressources à compléter.
iam.tf	Le rôle et la policy IAM en moindre privilège, dans un fichier séparé de <code>main.tf</code> pour isoler la gouvernance/sécurité du reste. C'est ce rôle que vous assumerez (<code>sts assume-role</code>) pour ingérer et interroger les données, plutôt que vos identifiants admin. Entièrement à écrire.
variables.tf	Les paramètres réutilisables du projet (nom, région, email d'alerte, budget) — pour ne jamais coder une valeur en dur. Déjà fourni dans le squelette.
outputs.tf	Les informations à récupérer après un <code>terraform apply</code> (nom du bucket, base Glue, ARN du rôle) — réutilisées pour l'ingestion et les requêtes Athena. Déjà fourni.
terraform.tfvars	Vos valeurs concrètes (email, région, budget) — jamais dans le code versionné. À créer depuis <code>terraform.tfvars.example</code> .

6. Étapes du projet (≈ 5 jours)

Jour 1 — Provisioning Terraform

Bucket S3 (bronze/silver/gold), rôle IAM, AWS Budget avec alerte, alarme CloudWatch. `terraform plan / apply`. Pas de workgroup Athena dédié à créer — le workgroup `primary` par défaut suffit (cf. indices).

Jour 2 — Ingestion & cataloguage bronze

Dépôt du CSV et des fichiers JSON dans `bronze/`, partitionnés par date d'ingestion, via le rôle IAM créé. Puis `CREATE EXTERNAL TABLE` Athena sur les données brutes et requêtes exploratoires (volumétrie, valeurs manquantes, doublons, incohérences de types).

Jour 3 — Silver + conception du modèle dimensionnel

CTAS vers silver/ (nettoyage, typage, Parquet partitionné). Conception du schéma en étoile : grain de la table de faits, dimensions nécessaires (date, client, produit) et leurs clés.

Jour 4 — Construction gold & livraison

CTAS pour générer fact_ventes et les dim_*. Requêtes BI finales (jointures fact/dimensions) répondant à la question métier. Export et documentation.

Jour 5 — Buffer

Vérification de la cohérence des clés entre faits et dimensions, terraform destroy propre, préparation de la restitution.

7. Critères d'évaluation

Critère	Ce qui est évalué
Terraform	Infrastructure 100% as code, aucune action manuelle console, code lisible et réutilisable.
IAM	Le rôle projet est réellement en moindre privilège (testé, ni trop large ni bloquant).
Budget / monitoring	Alerte Budget et alarme CloudWatch fonctionnelles, pas seulement déclarées.
Logique médaillon	Bronze jamais modifié, silver traçable depuis bronze, gold dérivé de silver uniquement.
Modèle dimensionnel	Grain de la table de faits clairement défini et respecté, dimensions non redondantes, clés cohérentes entre faits et dimensions (pas de clé orpheline non traitée).
Qualité de données	Les contrôles mis en place détectent réellement les anomalies présentes dans les données.
Résultat métier	Exactitude et pertinence des requêtes BI finales par rapport aux 6 questions analytiques.
Documentation	Le projet est reproductible par un tiers via un simple terraform apply.

Bon courage — traitez ce projet comme une livraison réelle à une équipe métier, pas comme un exercice technique isolé.